

# Testing Quick Reference

Where to find each new feature, the role you need, how to exercise it, what to expect, and the gotchas to watch for. Built for the local test stack.

## TEST ENVIRONMENT

**http://192.168.254.50:8080**

Log in through Keycloak. After any UI update do one hard refresh (**Ctrl-Shift-R**) the first time.

## TEST ACCOUNTS

**testadmin** TestAdmin12!!  
Administrator · team-alpha

**analyst2** Analyst2Pass!!  
Reader/Editor/Assessor · team-bravo

Role needed: **Admin** Administrator only **Editor+** Editor / Assessor / Admin **Any role** Reader and up

## ACCESS & FOUNDATIONAL

### 01 Package Access (RBAC)

**Admin**

**Where:** System Packages → open a package → **Update System** → “Package Access” section

Restrict a system package so only chosen Keycloak teams (or usernames) can see and use it. Unrestricted packages stay visible to everyone. Enforced across every fork service.

- 1 As **testadmin**, open a package and its Update System modal; tick **Restrict this package** and enter `team-alpha`, then Save Access.
- 2 Log in as **analyst2** (team-bravo) in a separate/private window.
- 3 Check the System Packages list, and try ConMon, Inventory, POA&M, Tasks for that package.
- 4 Back as testadmin, verify the same package still opens normally.

**EXPECT** analyst2 does **not** see the restricted package in any list/dropdown and gets “forbidden” (HTTP 403) on direct access; testadmin (admin) sees everything. Set it back to unrestricted and analyst2 can see it again.

**GOTCHA** A just-restarted service can briefly 403 on the first cold request before its checklist-DB read warms — retry once. Admins and internal service accounts always bypass RBAC by design.

## 02 Container Security (Trivy)

[Editor+](#)

Where: sidebar → Container Security

Runs the built-in Trivy scanner against a mounted folder (host path or NFS share), stores results with the same delta/trend engine as Nessus, and shows scan history, trends, and open findings.

- 1 Pick a **System Package** to attach results to.
- 2 Click **Browse** and traverse the mounted share to choose the folder to scan.
- 3 Click **Scan Now**; wait, then **Refresh**.
- 4 Review the run log, the Scan History & Trend table, and Open Findings.
- 5 Change a dependency file in the folder and scan again to see a delta.

**EXPECT** A “success” run row with C/H/M/L counts; findings listed by target and `CVE|package`. A second scan shows new/resolved/persisting deltas and a second trend snapshot.

**GOTCHA** The very first scan downloads Trivy’s vuln DB (slower); later scans take seconds. Narrowing the target folder marks now-out-of-scope findings “resolved” — keep a stable target per package for clean trends.

### 03 Scan Watch (automated ingest)

Admin

Where: sidebar → Scan Watch

Point STOOGE at an SMB share (or mounted folder) your cron job fills with scans; it auto-ingests new SCC XCCDF, `.ckl`, and `.nessus` files.

- 1 Enter the file server, share, and a **read-only** account; click **Test Connection** to preview the folders.
- 2 Save & enable. Drop a file into a subfolder named exactly like a system package.
- 3 Wait one poll cycle, then review the ingest log on the page.

**EXPECT** Test Connection lists the system folders; dropped files appear in the ingest log as “success” and show up on the matching package (checklists, trends, POA&M).

**GOTCHA** The subfolder name must match the system package **title** exactly (case-insensitive). Files newer than the “settle time” are skipped until they stop changing. Identical file content ingests only once.

### 04 Vulnerability Trends

Any role

Where: sidebar → Reports → Vulnerability Trends

Charts a system’s Nessus scan history — severity totals over time and new/resolved/persisting deltas per scan.

- 1 Choose a system package that has more than one Nessus scan.
- 2 Read the trend chart and the per-scan delta values.

**EXPECT** One point per scan in chronological order; deltas that reconcile against the prior scan (findings keyed by host + plugin).

## 05 Continuous Monitoring (ConMon)

Any role

Where: sidebar → **ConMon** · policy editing is **Admin**

SP 800-137 posture per system: scan/checklist freshness, open-finding aging, POA&M schedule health, asset coverage gaps, and STIG benchmark currency — with a cadence policy you set.

- 1 Read the portfolio table (freshness badges, overdue counts, coverage gaps, outdated STIGs).
- 2 Click a system's detail to see coverage host lists, aging buckets, and benchmark currency.
- 3 As admin, edit the **Cadence Policy** (e.g. patch-scan every 1 day) and Save; watch the freshness badge change.

**EXPECT** Green/amber/red freshness against your policy; coverage lists the right scanned-but-unchecklisted and checklisted-but-unscanned hosts; a shorter cadence flips a fresh scan to amber/red.

**GOTCHA** Some checklists show benchmark currency "unknown" when the STIG short title doesn't match a template (e.g. Apache 2.2, RHEL 7) — expected until the title-mapping table is enriched.

## 06 System Inventory

Any role

Editor+ to edit

Where: sidebar → **Inventory** · OS mapping rules are **Admin**

Every host the platform has seen, classified into OS classes (Windows 11, RHEL 8, ...) with fleet findings — "this CVE affects N of M RHEL 8 assets."

- 1 Choose a system; click the OS-class chips to filter the asset list.
- 2 Pick a class to open the **findings-by-class** panel.
- 3 Edit an asset (pencil) to set its OS/type manually — it gets a lock icon.
- 4 As admin, add/edit an **OS normalization mapping** rule; click **Reconcile Inventory**.

**EXPECT** Assets grouped by class with counts; fleet findings show affected/total hosts; a manually classified (locked) asset survives Reconcile; a new mapping rule reclassifies matching hosts.

**GOTCHA** Systems whose scans were uploaded before OS-capture need one scan re-upload (or the **Reconcile** button) to populate OS from scan data; checklist-based classification works immediately.

## 07 Live POA&M

Editor+

Where: sidebar → POA&M

A live Plan of Action & Milestones auto-built from findings, with the merge rule that never clobbers your edits, plus XLSX export.

- 1 Choose a system and Load POA&M; filter by status.
- 2 Edit an item (mitigations, status, milestones) and Save.
- 3 Re-upload a scan for that system, then reload the POA&M.
- 4 Export XLSX.

**EXPECT** Auto-created draft items for open findings; your hand edits survive reconciliation while automation still updates “last seen”; resolved findings move to “Completed – Pending Verification” (never deleted). XLSX opens with the records.

## 08 Review Tasks

Editor+

Where: sidebar → Tasks

Assign checklist / vulnerability / POA&M reviews to users with priority, due date, status workflow, comments, and full change history.

- 1 New Task → pick a system, target a checklist (picker), set assignee = `analyst2`, priority, due date.
- 2 Switch a task’s status; add a comment.
- 3 Log in as **analyst2** and open **My Tasks**.
- 4 Set a due date in the past to check overdue highlighting.

**EXPECT** The task appears in analyst2’s My Tasks; every status/field change is recorded in the task history with who/when; a past due date shows a red overdue flag. A status-only edit leaves other fields untouched.

## 09 Journal & Evidence

Where: `/api/journal/` — no dedicated UI page yet

An append-only change journal (who/what/when) built from platform events, plus user notes and file evidence attachable to any entity. Observable via the API; every action you take above also lands here.

- 1 Do any action (upload, POA&M edit, scan) as testadmin.
- 2 Query `GET /api/journal/journal/system/{systemGroupId}` and `/journal/activity` with a bearer token.
- 3 Optionally POST a note or an evidence file, then download the evidence back.

**EXPECT** Activity entries attributed to the acting user; evidence round-trips byte-for-byte with a recorded SHA-256.

Covers features added in the STOOGE fork of OpenRMF OSS. All fork services run on the UBI8 (RHEL 8.10) base. Test data and helper scripts live in

d:\OpenRMF\_Build\test-artifacts